Beta Testing Report

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Beta Testing Report

Transition-phase deliverable (UP): how the system was beta tested with real usage across all three roles, what that testing found, and the performance and security work that followed from it. Companion to TESTING.md (the **automated** test suite — 275 tests) and DEFECT_LOG.md (the defects this phase of testing actually found).

---

1. Approach

Automated tests prove the *application logic* is correct in isolation (see TESTING.md); they cannot tell you whether a real person, on a real phone, on real hostel Wi-Fi, can actually get a fault reported and fixed. Beta testing closed that gap in two ways:

1. **Structured UAT walkthroughs** — the student → administrator → technician flow, and every secondary feature, run end to end by hand against the deployed system, at both phone and desktop widths.
2. **Real usage by the project's own stakeholder** acting as an end user (not just reviewing code) — which is precisely how DEF-03 through DEF-08 in DEFECT_LOG.md were found: not by reading the source, but by using the deployed app the way a student actually would (on real, sometimes-unreliable connectivity) and reporting what felt wrong.

2. UAT scenarios and results

#	Scenario	Role(s)	Result
1	Register as a student, then log in with the ID number (not email)	Student	✅ Pass
2	Submit a request with a category, hostel, priority and a camera-captured photo	Student	✅ Pass
3	See an AI/rules triage suggestion while typing; a hazard phrase forces Critical regardless	Student	✅ Pass
4	See "Already reported" for a matching issue and back it with "Me too" instead of duplicating	Student	✅ Pass
5	New request appears on the administrator's dashboard **without a page refresh**	Student → Administrator	✅ Pass (via SSE live update)
6	Assign the ★ recommended technician in one tap	Administrator	✅ Pass
7	Student's status updates live to "Assigned"	Administrator → Student	✅ Pass
8	Post a comment as the student; it is visible to the assigned technician and an administrator, and to no one else	Student, Technician, Administrator	✅ Pass
9	Technician starts the task, attaches a photo and note, marks it resolved	Technician	✅ Pass

#	Scenario	Role(s)	Result
10	Student sees "Work Completed" with the photo/note and rates the repair	Student	✅ Pass
11	A request left unassigned/unresolved past its SLA target shows as overdue and alerts administrators + the assigned technician	Administrator, Technician	✅ Pass
12	Reports & Analytics: heatmap, technician performance table, "Needs a Permanent Fix" all show correct, consistent numbers	Administrator	✅ Pass
13	Post an announcement to one hostel; it appears as a banner for a student in that hostel and **not** for one in a different hostel	Administrator, Student	✅ Pass
14	A technician sees every active announcement regardless of hostel	Technician	✅ Pass (after DEF-07 fix)
15	Create a preventive-maintenance schedule, "Run now", confirm it flows through assignment and resolution exactly like a reported fault	Administrator, Technician	✅ Pass
16	Submit app feedback as a student; it appears on the administrator's App Feedback page with the correct average rating	Student, Administrator	✅ Pass
17	Toggle dark mode; every screen re-themes correctly	All	✅ Pass
18	Draft a request with Wi-Fi off; it queues; reconnect and confirm it submits automatically with no user action	Student	✅ Pass
19	Reload the installed app while briefly offline; confirm the session survives instead of signing out	Any	✅ Pass (after DEF-04 fix)
20	Install the app to the phone's home screen and confirm push notifications arrive	Student, Technician	✅ Pass

All 20 scenarios pass as of this report. Scenarios 14 and 19 are listed **as they behaved after** the fixes in DEFECT_LOG.md DEF-07 and DEF-04 respectively — both were *found* by an earlier run of this same beta-testing process, which is the point of doing it.

3. Defects found during beta testing

Every defect in DEFECT_LOG.md tagged "Found: Transition" (DEF-03 through DEF-08, six of the log's eight entries) was found through this beta-testing process, not through code review or automated tests — concrete evidence that manual UAT is finding a different, real class of issue (misleading messages, a role that was overlooked, a confusing label) than the automated suite is built to catch. This is the expected division of labour between the two: automated tests are cheap, fast and exhaustive over *logic*; beta testing is slow, human and exhaustive over *experience*.

4. Performance tuning

Prompted directly by using the deployed system on a throttled/mobile connection during beta testing, not designed speculatively up front:

- **Code splitting** — every page is its own lazily-loaded chunk; the entry bundle is ≈ 79 kB gzipped, and the ≈ 104 kB chart library loads only for administrators, who are the only role that uses it.
- **Stale-while-revalidate caching** (`apiCache.js`) — a revisited page renders instantly from the last-seen data while a fresh copy loads in the background, instead of a blank "Loading…" state on every navigation.
- **Image compression before upload** — a 4–10 MB phone photo is compressed client-side (max 1600 px, JPEG) before it ever reaches the network, turning it into a few hundred kB, specifically because beta testing on real hostel data made the uncompressed upload time obviously unacceptable.
- **gzip on API responses** and **cache-first service-worker strategy for hashed build assets** — full detail in ARCHITECTURE.md §8.
- **A scheduled health-check ping** (UptimeRobot → `/api/health`) added after beta testing surfaced how slow the first request after Render's free-tier idle sleep actually felt to a real user.

5. Security review and fixes

- **Row Level Security reviewed against the service layer's own ownership rules**, table by table, to confirm the two actually agree (RLS as a second line of defence, not the only one — see ARCHITECTURE.md §6).
- **The community "already reported" feed was specifically checked for identity leaks** — confirmed the response shape itself carries no name or ID field, backed by an integration test, not just a UI check (see R9 in RISK_REGISTER.md).
- **AI-prompt injection consideration**: a request description is user-controlled text that gets built into a prompt sent to Claude; it is treated as untrusted data and the safety floor (forcing hazards to Critical) is applied in code **after** the model responds, so a manipulated description cannot talk the model out of a safety-critical classification.
- **DEF-05's documentation-rendering fix** is, narrowly, a documentation bug rather than an application security issue — but the underlying lesson (untrusted-looking text can be silently dropped or misinterpreted by a renderer) is the same category of defensive thinking applied everywhere user input reaches HTML: email templates HTML-escape user-controlled text, and React escapes rendered output by default.
- **Hardening identified but intentionally not yet built**, stated plainly rather than silently deferred: restricting CORS to the deployed front-end origin (currently open), request-rate limiting on the login endpoint itself, and enabling Supabase's email-confirmation step — all listed in TECHNICAL_DOCUMENTATION.md §12.

6. Beta feedback going forward

Beta testing for this project was necessarily developer-led — there was no independent cohort of students to recruit for a final-year project's timeline. What **was** built, specifically to not let that limitation become permanent, is the in-app **App Feedback** channel (see TECHNICAL_DOCUMENTATION.md §7.14): a star rating, a category and an optional comment, reachable from every student's and technician's menu, read by an administrator on its own page with a live average rating. Its entire purpose is to be this report's successor once the system has real users — an always-on beta-feedback mechanism instead of a one-off exercise, for exactly the population (a student with no direct line to the developer) who could not take part in this phase's testing.